Laborator 7: Clase interne (partea a II-a)

1 Responsabil

Gabriel Guțu-Robu, gabriel.gutu@upb.ro
Publicat: 20 noiembrie 2022, 9:13
Actualizat la: 21 noiembrie 2022, 0:52 (eliminat Excepții, adăugat exerciții specifice claselor interne)

2 Clase interne în metode

O clasă se poate defini înăuntrul unei metode astfel:

```
class Extern {
     private String message = "Extern";

     void metoda() {
          class Intern {
               public void afiseazaExtern() {
                    System.out.println("message = " + x);
               }
          }
          Intern in = new Intern();
          in.afiseazaExtern();
     }
}
```

- Clasa `Intern` trebuie instanțiată **în metoda în care este definită**, deoarece vizibilitatea ei este la nivel de metodă, la fel ca a oricărei alte variabile declarate în metodă.
- O clasă internă poate accesa orice membru al clasei externe.
- O clasă internă NU poate accesa variabilele locale ale metodei în care este definită! Variabilele locale sunt salvate pe stivă și există atâta timp cât există și metoda. Când execuția metodei se încheie, stiva este eliberată. Se poate întâmpla ca atunci când metoda ia sfârșit, obiectul din clasa internă să mai existe pe heap. Din cauza diferențelor de lungime de viață, instanțele claselor interne nu pot accesa variabilele locale ale metodelor în care au fost definite. Acest lucru este posibil doar dacă se folosește modificatorul `final`. Motivul pentru care nu pot accesa o variabilă locală nefinală este că instanța locală a clasei rămâne în memorie după încheierea metodei. La încheierea metodei, variabilele locale devin *out-of-scope*, deci ar fi nevoie de o copie a lor. Dacă variabilele nu ar fi finale, copia lor din metodă s-ar

putea modifica, pe când cea din instanța locală a clasei nu, ajungând astfel la o desincronizare a valorilor variabilelor.

```
class Extern {
     private String x = "Extern";

     void metoda() {
          final String z = "variabila locala";

          class Intern {
               public void afiseazaExtern() {
                    System.out.println("x = " + x);
                    System.out.println("z = " + z);
               }
          }
     }
}
```

La fel ca orice variabilă locală, clasele definite în metode **NU** pot avea modificatorii `public`, `private`, `protected`, `static` etc. Se pot folosi doar `abstract` și `final`, dar nu amândoi în același timp.

O clasă definită într-o metodă statică are acces doar la membrii statici ai clasei externe.

3 Clase interne anonime

O clasă internă anonimă este o clasă fără nume. Clasele interne anonime pot fi definite atât în metode, cât și în argumentul unei metode.

```
class A {
     public void metoda() {
          System.out.println("Aceasta este o metoda");
     }
}
class B{
     A a = new A() {
          public void metoda() {
               System.out.println("metoda anonima");
          }
     };
}
```

În clasa `B` este creat un obiect de tipul `A` care nu are nume de clasă, dar este o subclasă a lui `A`. Aceasta este o *declarare anonimă*. Metoda `metoda()` este suprascrisă în definiția anonimă. Tocmai acesta este scopul claselor anonime – *de a suprascrie metode ale claselor inițiale sau de a implementa metode ale unei interfețe*. Definiția noii instanțe trebuie să se încheie obligatoriu cu semnul de punctuație “;”.

O implementare anonimă a unei interfețe se poate face astfel:

```
interface A {
     public void metoda();
}
class B {
     A c = new A() {
          public void metoda() {
               System.out.println("implementare anonima");
          }
     };
}
```

- O clasă internă anonimă poate implementa o singură interfață.
- O clasă internă anonimă nu poate extinde o clasă și implementa o interfață în același timp!
- Dacă o clasă internă anonimă este o subclasă a unei clase, automat devine o implementare a unei interfețe implementate de superclasă.

Pentru interfața `Runnable`, următorul exemplu este greșit:

```
Runnable r = new Runnable();
```

Este corectă implementarea interfeței într-o instanță anonimă:

```
Runnable r = new Runnable() {
     public void metoda() { }
};
```

3.1 Clase interne anonime ca argument

O clasă internă anonimă poate fi definită ca argument al unei metode:

```
class C {
     void exemplu() {
          B b = new B();
          b.executa(new A() {
               public void metoda() {
                    System.out.println("Execut metoda");
               }
          });
     }
}
interface A {
     void metoda();
}
```

```
class B {
     void executa(A a) { }
}
```

4 Moștenirea claselor interne

La moștenirea claselor interne, constructorul clasei derivate trebuie să se atașeze de un obiect al clasei externe.

```
class Extern {
     class Intern {
          public void metoda() {
               System.out.println("Clasa interna");
          }
     }
}

class Subclasa extends Extern.Intern {
     Subclasa(Extern ext) {
          ext.super();
     }
}

public class Test {
     public static void main(String[] args) {
          Extern ext = new Extern();
          Subclasa s = new Subclasa(ext);
          s.metoda();
     }
}
```

5 Aplicații

5.1 Test rapid de gripă și covid

În plin sezon de gripe (de tip A și tip B) și COVID, aveți nevoie să implementați o soluție rapidă pentru detectarea tipului de gripă în funcție de secreția nazală colectată.

În cadrul acestui link[1] vă sunt puse la dispoziție următoarele fișiere:

- `Mucus.java` – conține clasa Mucus, care este folosită pentru instanțierea obiectelor în funcție de tipul gripei și un „procent de siguranță” (nu modificați acest fișier);

[1] https://github.com/ACS-POO-2CB/lab-public/tree/main/poo-lab07/src/inner_class/inside_method

- `InfluenzaDetector.java` – conține clasa InfluenzaDetector ce are o singură metodă checkAllFlyTypes, care primește ca parametru un obiect de tip Mucus. Metoda încearcă instanțierea unor obiecte de tipul InfluenzaA, InfluenzaB și COVID, iar apoi afișează rezultatul prin apelul unor metodei isInfected apelată pe obiectele respective. Trebuie să adăugați înăuntrul metodei checkAllFlyTypes cele 3 clase interne corespunzătoare, împreună cu metodele aferente lor.
- `Main.java` – instanțiază un obiect InfluenzaDetector și rulează mai multe teste. Puteți modifica fișierul pentru a include teste suplimentare.

5.2 Manager de studenți cu conexiune la baza de date

Aveți nevoie de integrarea unui manager de studenți cu o bază de date. Pentru aceasta trebuie să respectați un template impus într-o interfață.

În cadrul acestui link[2] vă sunt puse la dispoziție următoarele fișiere:

- `IDatabase.java` – interfața template ce definește metode de connect și disconnect (ce nu primesc parametru) și, respectiv insert, update și delete (ce primesc un parametru de tip Object) – nu modificați acest fișier;
- `Student.java` – o clasă simplă ce poate instanția studenți cu nume și având o reprezentare de tip toString – nu modificați acest fișier;
- `StudentManager.java` – o clasă ale cărei obiecte vor stoca un obiect de tip IDatabase. Clasa conține un ArrayList de obiecte de tip Student și metode pentru insert, update și delete. Fiecare metodă modifică lista și comunică cu baza de date. De asemenea, clasa conține și o reprezentare de tip toString. La finalul clasei există și o metodă runSomeTests(), pe care o puteți modifica pentru a adăuga teste;
- `Main.java` – conține un schelet de cod pentru apelul metodei runSomeTests() pentru un manager de studenți bazate pe implementări concrete (MySQL și NoSQL). Adăugați cele necesare (implementarea unor clase anonime) astfel încât să faceți codul să compileze.

5.3 (Bonus) Manager de studenți cu HashMap

Modificați clasa StudentManager astfel încât să folosească un HashMap în loc de un ArrayList.

[2] https://github.com/ACS-POO-2CB/lab-public/tree/main/poo-lab07/src/inner_class/anonymous